A bootstrap ensemble of `HEFTNet` models

Plan for `heft_train_ensemble.ipynb` and `dihiggs/ensemble.py`

For approval before any code is written

October 30, 2026

Abstract

The first Bayesian neural network run (`docs/bnn/bnn_run1.pdf`) gave credible bands about 17 times narrower than the model's actual error, partly because a single HMC chain explores a single valley of the posterior. This plan describes a complementary, frequentist approach: train many copies of `HEFTNet`, each on a *bootstrap* resample of the training spectra and with its own random seed, and use the spread of their predictions as an estimate of the standard error of the trained network. Two ensembles are proposed: one that varies only the seed (initial weights and batch order) on the full data, and one that also resamples the data. Together with the BNN they separate the uncertainty due to the optimisation (which valley), the uncertainty due to the finite training sample, and the bias that neither captures. The work is a new notebook, `code/heft_train_ensemble.ipynb`, and a small new module, `dihiggs/ensemble.py`. Nothing will be written until you approve; the decisions needed are collected in section 10.

The proposal in brief

1. **Two ensembles** of $M = 20$ members each (table 1): `'seeds'` (full data, different seeds) and `'bootstrap'` (resampled spectra, different seeds).
2. **Resample whole spectra,** never individual rows: the 80 bins of a spectrum share θ, a Monte Carlo run and the model's misfit, so they are not independent.
3. **Each member is trained exactly as in** `heft_train.ipynb` (L1 loss, Adam, the same learning-rate steps), and saved as an ordinary run folder, so that `heft_test.ipynb` and `heft_export.ipynb` work with any member.
4. **The validation spectra are chosen once,** by spectrum, and are the same for every member, so no copy of a training spectrum is ever used for validation.
5. **An** `Ensemble` **class** with the same methods as `hmc.BNN` (`mean`, `std`, `quantiles`, `interval`, `apply`).
6. **The test** is on the 1,000 holdout spectra: is the ensemble spread, added to the POWHEG noise, the size of the actual differences from POWHEG?

Contents

1 Why an ensemble 2

2 What the ensemble estimates **2**
2.1 The frequentist view 2
2.2 Separating the two sources 3
2.3 How many members? 3

3 Resampling spectra, not rows **4**

4 Training the members **4**

5 The module `dihiggs/ensemble.py` **5**

6 The notebook `heft_train_ensemble.ipynb` **5**

7 The test: is the spread the size of the error? **6**

8 Links with the BNN **7**

9 Not in this plan **7**

10 Decisions for you **7**

1 Why an ensemble

The BNN run showed three reasons why a single chain's band is too narrow (`bnn_run1.pdf`, §6): incomplete exploration of the valley it is in; other valleys, in which equally good networks give different predictions where the data constrain them weakly; and the model's own systematic error, which no spread of parameters within one model describes. An ensemble of independently trained networks addresses the second directly. Each training run, started from different random weights and fed batches in a different order, ends in a different valley. Large studies have found that such "deep ensembles" reproduce the predictions of long, full-data HMC better than most other approximations, precisely because they visit several valleys [4, 3, 2].

Resampling the training data as well, by the bootstrap [1], adds the second source of variation that any estimator has: the training set is one finite sample, and a different sample would give a different network.

2 What the ensemble estimates

2.1 The frequentist view

Think of the whole training procedure as an *estimator*: it takes a training set D and a random seed s and returns a function $\hat{f}(\boldsymbol{x}; D, s)$. Had POWHEG been run again, D would be different; had the seed been different, the optimiser would have ended elsewhere. So $\hat{f}(\boldsymbol{x})$ at a fixed point $\boldsymbol{x}$ is a random variable, with a mean and a variance. Its standard deviation is the *standard error* of the trained network at $\boldsymbol{x}$, the frequentist counterpart of the width of a Bayesian band.

The bootstrap principle

We have only one training set, so the distribution of $\hat{f}$ over training sets cannot be observed directly. The bootstrap [1] replaces the unknown population of spectra by the sample we have: a bootstrap training set D^* is made by drawing n spectra *with replacement* from the n available. On average a bootstrap set contains $1 - e^{-1} \approx 63\%$ of the distinct spectra, some of them more than once. The spread of $\hat{f}(x; D^*_m, s_m)$ over $m = 1, \ldots, M$ estimates the spread of $\hat{f}$ over real training sets.

2.2 Separating the two sources

The law of total variance splits the variance of $\hat{f}$ at x into an optimisation part and a data part:

$$\underbrace{\mathrm{Var}_{D,s}\left[\hat{f}\right]}_{\text{total}} = \underbrace{\mathrm{E}_D\Big[\mathrm{Var}_s\left[\hat{f} \mid D\right]\Big]}_{\text{optimisation: which valley}} + \underbrace{\mathrm{Var}_D\Big[\mathrm{E}_s\left[\hat{f} \mid D\right]\Big]}_{\text{data: which sample}}. \tag{1}$$

The two ensembles of table 1 estimate the first term and the total. The data term is their difference, assuming the optimisation spread is about the same for a bootstrap set as for the full set.

`MODE`	training set of member m	seed	estimates	run folder
`'seeds'`	all training spectra	`SEED` $+m$	optimisation term	`runs/ENS_seeds`
`'bootstrap'`	bootstrap resample m	`SEED` $+m$	total	`runs/ENS_bootstrap`

Table 1: The two ensembles. Member m uses the same seed in both, so the two members start from the same initial weights; only the data differ.

What neither ensemble captures: the shared bias

All members have the same architecture, the same loss and the same training schedule. If they all miss the plateau just above threshold in the same way, they agree with each other, and the spread stays small however many members there are. The bootstrap standard error is an honest error bar on the *estimator* $\hat{f}$, not on the true cross section, unless the bias $\mathrm{E}[\hat{f}] - \sigma_{\text{true}}$ is small compared with it. Near threshold it may not be. The test of section 7 is designed to show whether it is.

2.3 How many members?

The standard deviation of M draws is itself uncertain, by a relative amount of about $1/\sqrt{2(M-1)}$:

M	10	20	50	100
relative uncertainty of the standard deviation	24%	16%	10%	7%

I propose $M = 20$ per ensemble, enough to tell a factor of two in width, which is the level of the questions we are asking. Members can be added later without redoing the others.

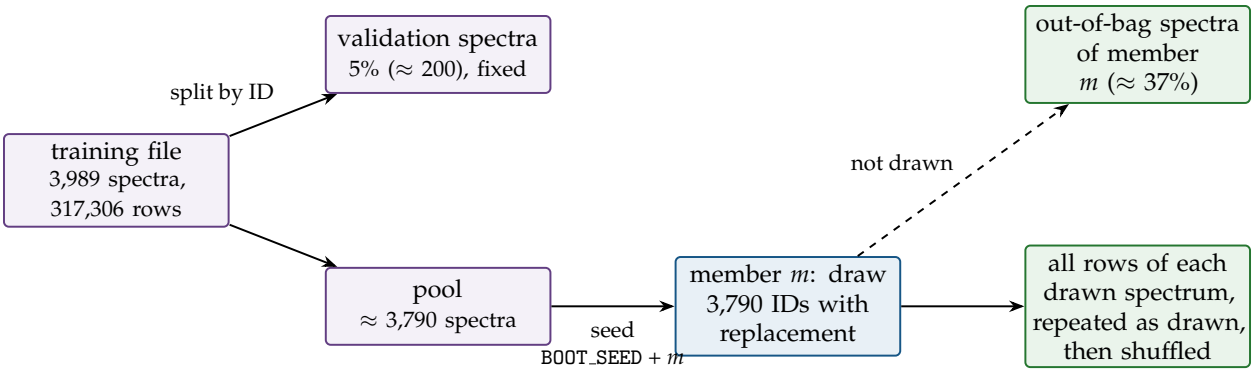


Figure 1: How the training set of one bootstrap member is formed. The validation spectra are set aside once, for all members and both ensembles.

3 Resampling spectra, not rows

- **Why spectra.** The 80 bins of a spectrum share θ and come from one POWHEG run, and the model's errors are correlated across them, most strongly near threshold. Resampling rows would treat them as 80 independent pieces of information and underestimate the spread, the same overconfidence that afflicts the BNN's likelihood. Resampling spectra is the "block" or "cluster" bootstrap.
- **The validation set.** `heft_train.ipynb` splits the training file by row, so a spectrum's bins are spread over training, validation and its quick-test rows. That is harmless for a single model, whose real test is the holdout file. In a bootstrap ensemble it would put copies of training spectra into validation, and the validation loss, which chooses the saved parameters, would be biased. So the new notebook sets aside 5% of the *spectra* once (seed `SEED`), for every member of both ensembles.
- **Shuffling.** The rows of a bootstrap set are built spectrum by spectrum, so they are shuffled before training. (`drwho`'s `DataLoader` reshuffles only after the first pass through the data.)
- **Bookkeeping.** Each member's folder gets `bootstrap_ids.csv`: the IDs drawn and how often. Together with `BOOT_SEED` this makes every member reproducible and allows the *out-of-bag* spectra, those a member never saw, to be used as a free extra test of that member.
- **Size.** A bootstrap set has as many spectra as the pool, so its number of rows, about 301,000, is close to that of the full set, and the training schedule needs no change.

4 Training the members

Each member is trained with the code of `heft_train.ipynb`, unchanged except for the data: L1 loss (as `nn.L1Loss`), Adam, batch size 2,048, 200,000 iterations in four learning-rate steps from 2×10^{-3}, parameters saved at the best validation loss. Before member m is built, `torch.manual_seed(SEED + m)` and `np.random.seed(SEED + m)` fix its initial weights and its batch order.

Run folders. Member m of an ensemble is saved in `runs/ENS_<mode>/member_01/`, `member_02/`, ..., each an ordinary run folder (`dihiggs_config.yaml`, `dihiggs_params.pth`, `dihiggs_losses.csv`), plus `bootstrap_ids.csv` for bootstrap members. The configuration records `MODE`, the member

number, both seeds and the training file's SHA-256. A summary file `runs/ENS_<mode>/ensemble.csv` has one row per member: seeds, number of distinct spectra, number of rows, best validation loss, out-of-bag χ^2/N, training time.

Interruption and parallel running. Members are trained one after another, and a member whose parameter file and summary row already exist is skipped, so an interrupted loop resumes where it stopped. A switch `MEMBERS` (e.g. `range(0, 10)`) chooses which members a notebook trains, so that two or more copies of the notebook can train disjoint sets of members at the same time on the 5090. `HEFTNet` is small; several trainings fit on one GPU at once.

The real-time loss monitor. `heft_train.ipynb` opens a window that plots the losses during training. In a loop over 20 members that would open 20 windows, so the ensemble notebook writes the loss files without starting the window (`monitor.start()` is not called); progress is printed instead.

Cost. About 42 minutes per member on a CPU; on the 5090 I expect a few minutes, but the notebook will time the first member and print an estimate for the rest. The `N_ITERATIONS` switch allows shorter members for a first trial.

5 The module `dihiggs/ensemble.py`

A class `Ensemble` with the same interface as `hmc.BNN`, so that analysis code can use either:

```
from dihiggs import ensemble
from dihiggs import heftnet as NN

ens = ensemble.Ensemble(NN.HEFTNet, 'runs/ENS_bootstrap')  # every member_*/
mu  = ens.mean(x)                  # ensemble mean, one value per row of x
sd  = ens.std(x)                   # spread of the members (the standard error)
lo, hi = ens.interval(x, cl=0.68)  # central 68% of the members
A   = ens.apply(lambda m, x: m.coeffs(x), xs)   # a_i(m_hh) for every member
```

It reads `member_*/dihiggs_params.pth` by default (any file pattern can be given), and also provides `summary()`, the contents of `ensemble.csv`. Like `hmc.py`, it imports nothing from `dihiggs`, so it can move to `drwho`.

Why not reuse `hmc.BNN` directly?

`BNN` already averages over a list of parameter files, and an ensemble needs the same arithmetic. The tidy design is a common base class (an ensemble of parameter files) of which `BNN` is a special case that also reads the noise scales. I propose to write `Ensemble` as that base class now, and to make `BNN` a subclass of it when both modules move to `drwho`, so that `hmc.py`, which works and has been checked, is not changed now (section 10).

6 The notebook `heft_train_ensemble.ipynb`

In the style of `heft_train.ipynb` (header, Colab cell with `USE_DRIVE`, imports, device, data with its SHA-256), then:

1. **Introduction** (markdown): the estimator view, the bootstrap, the decomposition (1), and why spectra are resampled.
2. **Switches:** `MODE = 'bootstrap'` or `'seeds'`; `N_MEMBERS = 20`; `MEMBERS = range(N_MEMBERS)`; `SEED = 42`; `BOOT_SEED = 1000`; `VAL_FRACTION = 0.05`; `N_ITERATIONS = 200_000`. `DIRNAME = ENS_<mode>`.
3. **The split by spectrum:** validation IDs and the pool, printed with a fingerprint of the validation IDs (as `heft_prepare_data` does for the holdout).
4. **One member's data:** a function returning the shuffled rows of member m and its table of drawn IDs.
5. **One member's training:** a function that builds the configuration, model, optimiser, scheduler and monitor, trains, and appends the summary row; the training loop is that of `heft_train.ipynb`.
6. **The loop** over `MEMBERS`, skipping finished members.
7. **Results on the holdout spectra** (section 7): the ensemble mean against POWHEG; spectra with ensemble bands; the spread against m_{hh}; the pull test; and, when both ensembles exist, the decomposition (1) against m_{hh}, and the comparison with the BNN run if there is one.

7 The test: is the spread the size of the error?

On the 1,000 holdout spectra, at every bin k with POWHEG value t_k, POWHEG noise σ_k (as in `heft_test.ipynb`), ensemble mean $\bar{f}_k$ and ensemble standard deviation s_k, form the pull

$$z_k = \frac{t_k - \bar{f}_k}{\sqrt{\sigma_k^2 + s_k^2}}. \tag{2}$$

If the ensemble spread describes the model's error, and the error and the POWHEG noise are independent, z_k has mean 0 and standard deviation 1, and $|z_k| < 2$ for about 95% of bins. If the spread is too small, the standard deviation of z is larger than 1. The same pull, with s_k the posterior standard deviation, is computed for the BNN, so all three are compared on one scale.

quantity	reported
accuracy of the ensemble mean	median χ^2/N; fractions above 2 and 5; mean χ^2 per bin at 255–555 GeV and above; against the single members and the L1 model
width	s_k/σ_k: median, distribution, against m_{hh}
calibration	standard deviation of z; fraction $\lvert z\rvert < 2$; both against m_{hh}
decomposition	optimisation and data terms of (1) against m_{hh}
out-of-bag (bootstrap only)	each member's χ^2/N on the spectra it did not see

What counts as success.

1. The ensemble mean is at least as accurate as a single L1 model, as averaging over members usually makes it.

2. The spread is *informative*: larger where the model is known to be weak (near threshold) than elsewhere.
3. The standard deviation of z is close to 1 above about 555 GeV. Near threshold I expect it to exceed 1, because of the shared bias; how much it exceeds 1 measures that bias, which is itself a useful number.

8 Links with the BNN

- **Starting points for several HMC chains.** The members of the `'seeds'` ensemble lie in different valleys. Starting four BNN chains from four of them makes $\hat{R}$ across chains a real test for several valleys, which it is not when every chain starts from the same L1 model. This needs a small option in `heft_train_bnn.ipynb`, `START_FROM`, a list of parameter files, one per chain, to be added after this plan.
- **A common yardstick.** The pull (2) puts the BNN band, the two ensemble spreads and the POWHEG noise on one scale.

9 Not in this plan

- No change to `heft_train.ipynb`, `heft_test.ipynb`, `hmc.py` or the data files.
- The `START_FROM` option of the BNN notebook (above), and the tempered likelihood discussed on October 30, are separate steps.
- Bootstrap confidence intervals more refined than the spread (percentile, BCa) are left out: with $M = 20$ only the standard deviation is well determined.
- Nothing is committed to git.

10 Decisions for you

Decisions before I proceed

1. **Two ensembles,** `'seeds'` and `'bootstrap'`, $M = 20$ each. Agreed, or start with 10?
2. **Validation by spectrum,** 5% of the training spectra, fixed for all members. Agreed?
3. **The training recipe** of `heft_train.ipynb`, unchanged (L1, 200,000 iterations). Agreed?
4. **No real-time monitor window** during ensemble training; printed progress instead. Agreed?
5. `Ensemble` **as a new module,** `dihiggs/ensemble.py`, leaving `hmc.py` unchanged for now. Agreed?
6. **Folder names:** `runs/ENS_seeds/member_01/`, ... Acceptable?
7. **Location of this document:** `docs/ensemble/ensemble_plan.pdf`?

References

[1] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall/CRC (1993).

[2] B. Lakshminarayanan, A. Pritzel and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles", NeurIPS 2017, arXiv:1612.01474.

[3] A. G. Wilson and P. Izmailov, "Bayesian deep learning and a probabilistic perspective of generalization", NeurIPS 2020, arXiv:2002.08791.

[4] P. Izmailov, S. Vikram, M. D. Hoffman and A. G. Wilson, "What are Bayesian neural network posteriors really like?", ICML 2021, arXiv:2104.14421.